CSE 554: Assignment 1

Jack Lowry, Nam Pho, Brett Saiki

January 28, 2026

1 Implementing the Sigmoid Linear Unit (SiLU)

Q1. Conceptual Understanding

(a) What are the use cases of SiLU in Large Language Models (LLMs)?

SiLU was shown to have improved accuracy over the original ReLU activation function and other (at the time) commonly used non-linearity functions [2]. SiLU is sometimes referred to as "swish" and it was combined with GLU resulting in the SwiGLU activation function [3]. Shazeer demonstrated that the SwiGLU function deployed within the transformer architecture improved model quality. SwiGLU was adopted as the primary transformer activation function in modern LLMs, such as Llama 3 [1].

(b) What is the mathematical formula for the SiLU function?

$$\mathrm{SiLU}(x) = x \cdot \sigma(x) = \frac{x}{1 + \exp(-x)} \tag{1}$$

Q2. PyTorch Implementation and Profiling Implement the SiLU function for an $(8192, 8192)$ matrix in `silu/silu_torch.py` using basic tensor operations (do not use `nn.SiLU` or `nn.Sigmoid`).

(a) Profile execution using Torch Profiler (`torch_silu.json`) and NVIDIA Nsight Systems (`torch_silu.nsys-rep`).

```
$ uv run python assignment1/silu/silu_torch.py
====================================================================
PyTorch SiLU implementation and performance profiling
====================================================================
[+] CUDA device: Quadro RTX 6000
[+] Tensor shape: torch.Size([8192, 8192])
[+] Running 100 warmup iterations...
[+] Exporting trace to torch_silu.json

====================================================================
Performance Numbers
```

```
=================================================================
[+] Time for SiLU: 4.438591957092285 ms
[+] Bandwidth: 120.95523021487728 GBps

=================================================================
Correctness Test
=================================================================
[+] Test closeness: PASS (<0.00e+00)

$
```

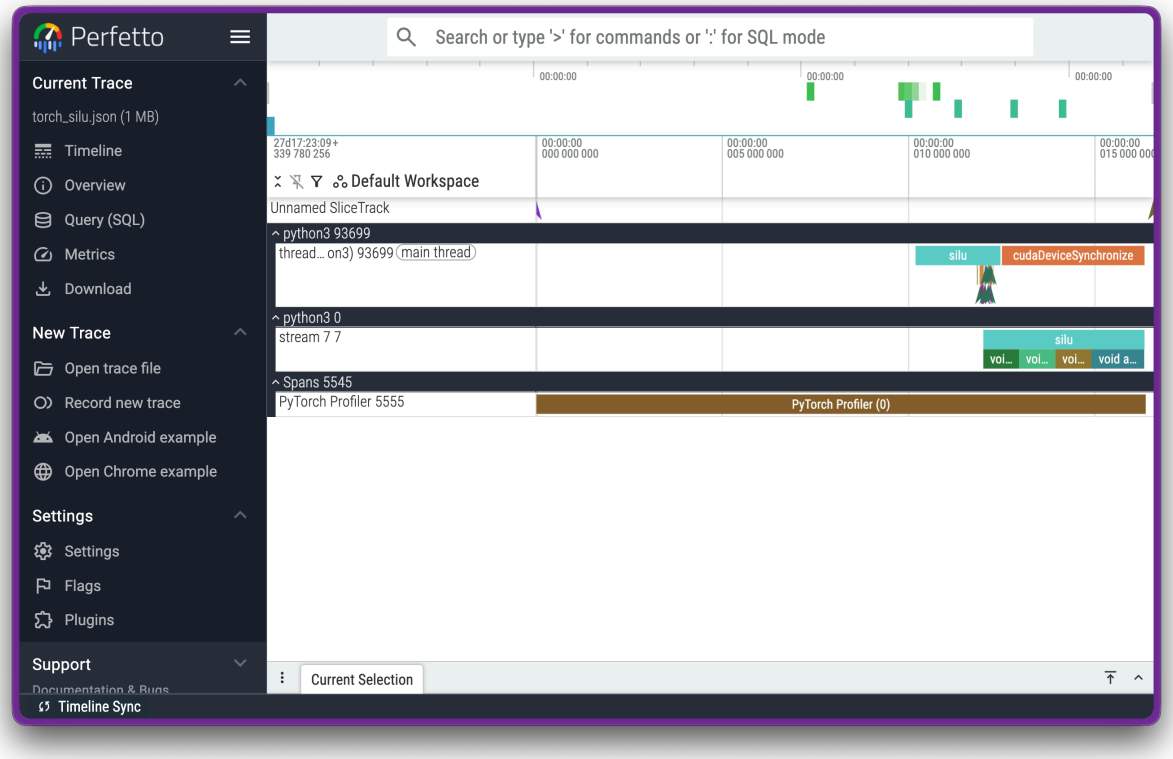

Figure 1: Performance profiling our SiLU implementation with the Torch profiler. The `torch_silu.json` trace visualized using ui.perfetto.dev.

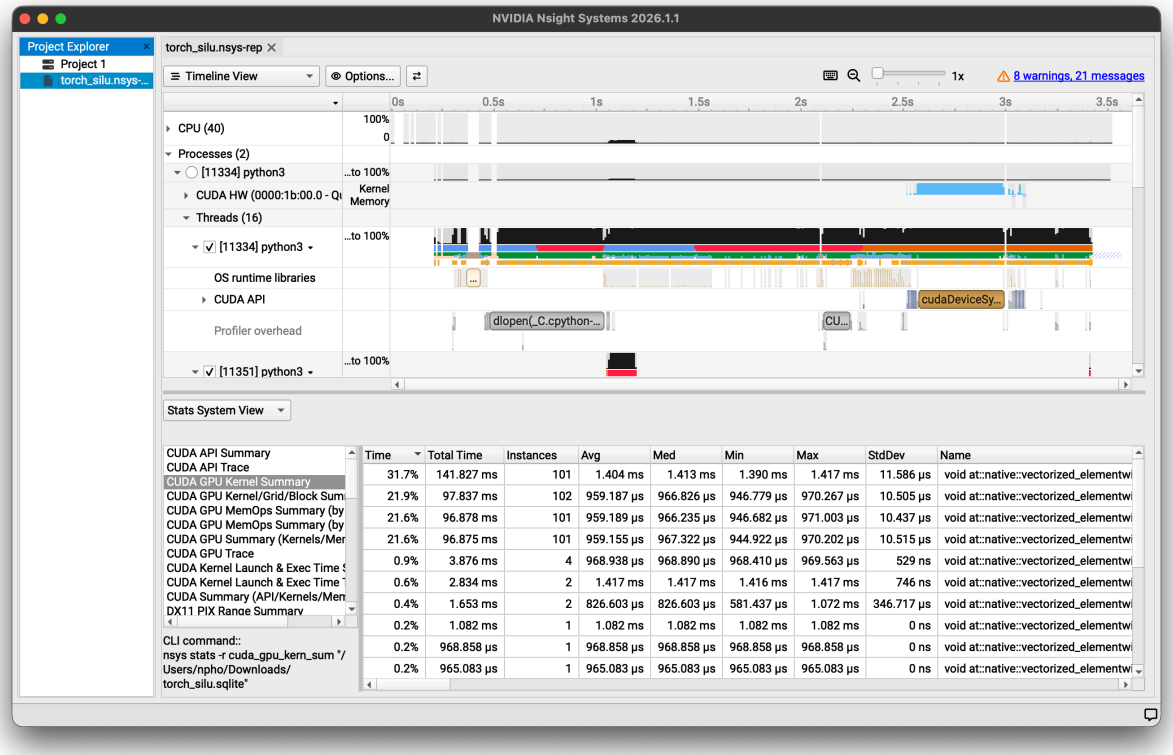

Figure 2: Performance profiling our SiLU implementation using NVIDIA Nsight Systems.

(b) i. What kernels are being launched?
There were 17 total non-unique kernel launches but the most notable ones are step applications for the SiLU function. For example, the SiLU argument is first negated with the `neg_kernel_cuda` kernel. Before then being added to a constant with the `CUDAFunctor_add` kernel, the negated argument is exponentiated with the `exp_kernel_cuda` kernel. There are other less clear cut kernel calls such as `BinaryFunctor`, `AbsFunctor`, `CompareFunctor`, `AUnaryFunctor`, `distribution_elementwise_grid_stride_kernel`, and `ReduceOp`.

ii. What is the minimum number (discount repetition and intermediate results) of memory accesses necessary (read and write) to accomplish the task?
There are **two memory accesses** necessary, <u>one load</u> and <u>one store</u>.

iii. What is the bandwidth usage (minimum number of necessary memory accesses / total execution time) of SiLU?

$$\frac{\#\ \text{Accesses} \cdot \text{Data Size}}{\text{Execution Time}} = \frac{2 \cdot 4 \cdot 8192 \cdot 8192 \cdot 10^{-9}}{4.43 \cdot 10^{-3}} \approx \boxed{121.28\ \text{GBps}}$$

Two memory accesses (i.e., one load and one store), for a tensor array of 8192×8192 elements with 32-bit floating point (i.e., 4 Byte) representation running for approximately 4.43 ms per function call.

Q3. Triton Implement the SiLU function using Triton in `silu/silu_triton_kernel.py`. Test the kernel and report the timing and bandwidth.

The Triton implementation runs in 1.0179 ms compared to 4.3323 ms for Triton, a **4.26 faster speed up** for the run time. The Triton implementation moves data at 527.45 GB/s compared to only 123.92 GB/s for PyTorch, a **4.32 times increased bandwidth**.

```
$ uv run python assignment1/silu/silu_triton_test.py
CUDA device: Quadro RTX 6000

====================================================================
Correctness Test
====================================================================
Tensor Shape         Test Status          Max Diff
--------------------------------------------------------------------
(1024, 1024)         PASS                 4.77e-07
(2048, 2048)         PASS                 7.15e-07
(4096, 4096)         PASS                 9.54e-07
(8192, 8192)         PASS                 9.54e-07

====================================================================
Runtime Benchmark
====================================================================
Size                 PyTorch (ms)    Triton (ms)     Speedup
--------------------------------------------------------------------
(1024, 1024)         0.0669          0.0511          1.31
(2048, 2048)         0.2829          0.0955          2.96
(4096, 4096)         1.0753          0.2772          3.88
(8192, 8192)         4.3323          1.0179          4.26

====================================================================
Bandwidth Benchmark
====================================================================
Size                 PyTorch (GB/s)  Triton (GB/s)   Speedup
--------------------------------------------------------------------
(1024, 1024)         125.34          164.25          1.31
(2048, 2048)         118.61          351.46          2.96
(4096, 4096)         124.82          484.14          3.88
(8192, 8192)         123.92          527.45          4.26

$
```

Q4. CUDA Implement the kernel in `silu/CUDA/silu.cu`. Use NVIDIA Nsight Compute

(ncu) for analysis (`cuda_silu.ncu-rep`). Optimize the kernel to achieve bandwidth utilization > 500 GBps.

The final run took **0.9720 ms** and utilized **552.35 GBps** of bandwidth.

```
$ ./silu
====================================================================
SiLU Kernel Tests
====================================================================
CUDA device:    Quadro RTX 6000
CPU time:       315.5518 ms
GPU time:       0.9720 ms
Speedup:        324.65x
Bandwidth:      552.35 GB/s
Max error:      1.91e-06
Status:         PASSED

$
```

2 Implementing RMS Norm

Q1. Conceptual Understanding What is RMS Norm? How is it used in LLMs? Provide the mathematical formula.

Root mean square layer normalization (RMSNorm) helps stabilizes training and boosts model convergence by re-centering and re-scaling activations [4]. Specifically, the activations are normalized by the root mean square (RMS) and then scaled by a learnable factor.

$$\text{RMSNorm}(\mathbf{x}) = \frac{\mathbf{x}}{\sqrt{\frac{1}{n}\sum_{i=1}^{n} x_i^2 + \epsilon}} \odot \mathbf{g} \tag{2}$$

In the equation above, $\mathbf{x}$ is an input vector of size n, ϵ is a small constant for numerical stability, and $\mathbf{g}$ is a learned scaling parameter (an element-wise multipled vector).

Q2. Matrix Kernel Implementation Implement a CUDA kernel for an $(8192, 8192)$ matrix. Optimize for bandwidth > 300 **GB/s** (`rms_norm_matrix.ncu-rep`).

The optimized kernel executes in **0.92 seconds** and achives **573 GB/s** of memory throughput according to Nsight Compute.

Q3. Vector Kernel Implementation Implement a CUDA kernel for a vector of shape $(1, 1024 \times 1024)$. Optimize for bandwidth > 200 **GB/s** (`rms_norm_vector.ncu-rep`).

The optimized kernel executes in **0.55 seconds** and achives **491 GB/s** of memory throughput according to Nsight Compute.

Q4. Thread Block Analysis Compare the thread block assignments for the matrix and vector kernels. Explain the differences and why they were necessary for performance.

For the matrix kernel, we mapped each row x_i to its own block; each block was responsbile for computing RMSNorm for the corresponding row of the matrix. For the vector kernel, allocation a single block resulted in poor utilization; we could only run as many threads a block could support at a time. The solution for higher throughput was to perform the root mean square computation via a reduction across many blocks, and then apply normalization across the blocks. This change allowed for much more parallelism.

3 Host–GPU Memory Copy

Q1. Bandwidth Measurement Measure Host-to-GPU and GPU-to-Host transfer speeds for data sizes 2^0 to 2^{20} bytes using C++. Plot the bandwidth vs. transfer size curve and identify the peak speed.

The max observed bandwidth was going host-to-device for 2^{20} bytes at **3.76 GB/s** (Figure 3).

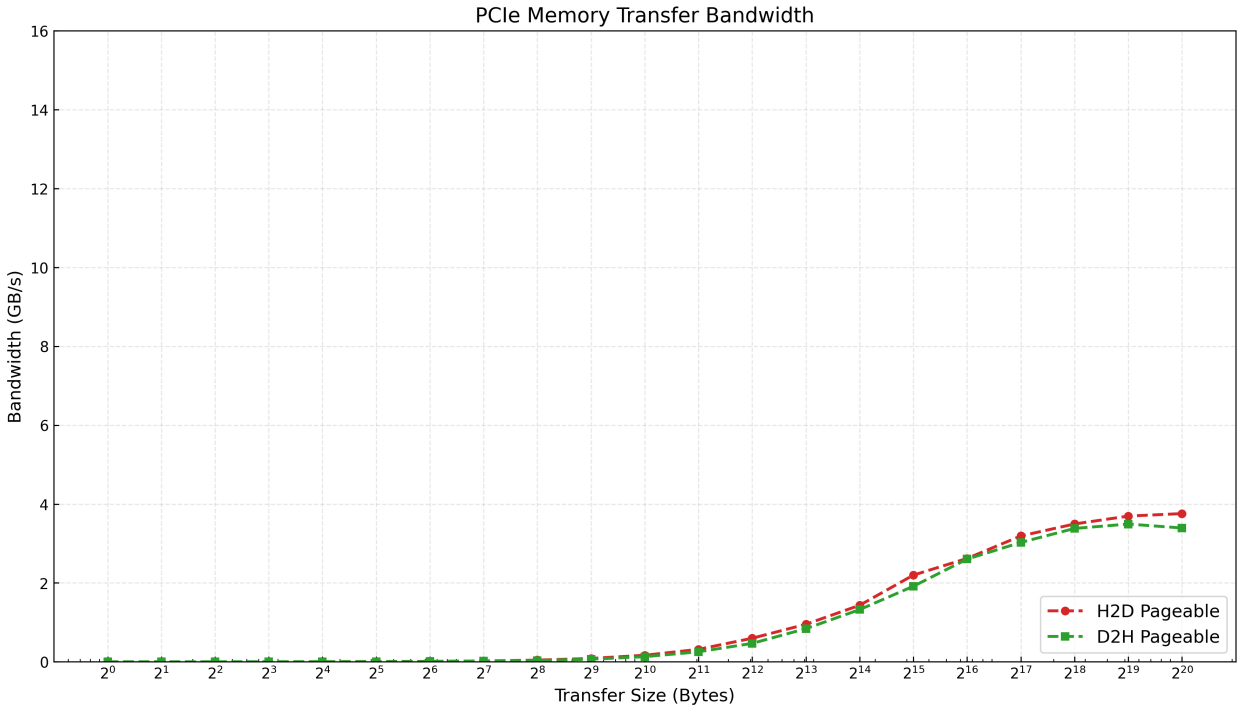

Figure 3: Exploring peak bandwidth (GB/s) as a result of different bi-directional transfer sizes between paged host memory and device (GPU) memory.

Q2. Pinned Memory Define pinned memory and explain its impact on CPU-GPU transfer performance. Add a pinned memory curve to the plot from Q1.

Pinned memory is considered page locked memory on the host device (i.e., CPU). In practice, this means the memory will not be swapped out in virtual memory to disk, ensuring it has priority and optimal performance. Pinned memory is also mapped to the same memory address space as the GPU. The new peak speed under pinned memory is now **12.03 GB/s** (Figure 4) for device-to-host transfer of 2^{20} bytes. This is a **3.547-times** speed up of the pinned versus paged memory version.

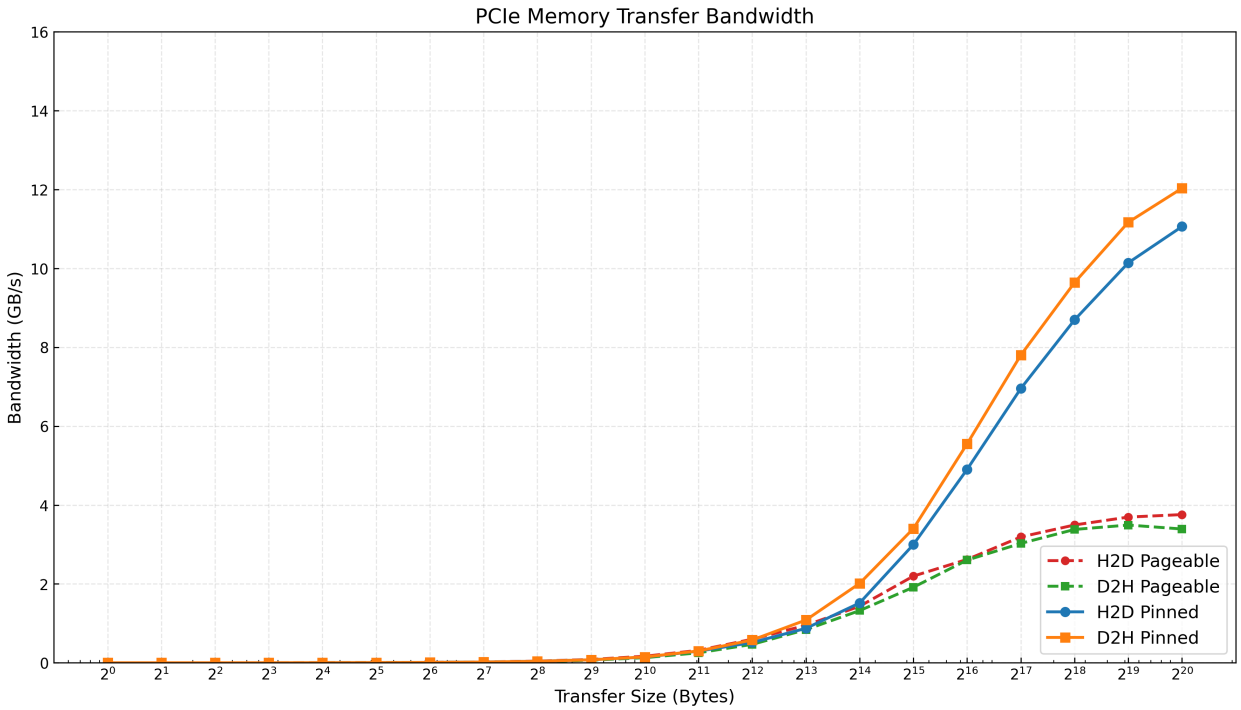

Figure 4: Repeating the data transfer experiment but with pinned memory and combining the plots.

Q3. Optimized Column Extraction Given an $(8192, 65536)$ matrix on the CPU (row-major), implement an efficient routine in `host_GPU/copy_first_column.cu` to copy **only the first column** of each row to the GPU. **Target:** Total transfer time must be $< 100\ \mu s$ (`copy.nsys-rep`).

```
$ nsys profile -o copy.nsys-rep ./copy

<<< CUT >>>

=====================================================================
Strided Column Copy Benchmark (copy_first_column)
=====================================================================
      Matrix: 8192 x 65536 floats (2048.000 MB)
      Column: 8192 floats (32.000 KB)
  Iterations: 100
```

```
Average time: 91.75 us (PASS)
    Accuracy: 0.00 (PASS)

Generating '/tmp/nsys-report-a2cf.qdstrm'
[1/1] [========================100%] copy.nsys-rep
Generated:
        /mmfs1/gscratch/scrubbed/npho/cse554/assignment1/host_GPU/
        build/copy.nsys-rep
$
```

The idea is that while using pinned memory may be an attractive idea in this case, the overhead to allocate pinned memory is high. In instances where the relative compute for a given data size is very high, it makes sense to take the pinned memory overhead. For example, the benchmarking done on pinned versus paged memory (Figure 4) demonstrated that the benefit of pinned over paged memory only occurs at larger data transfer sizes. For the smaller sizes we have the bandwidth difference is negligible but we still incur the overhead penalty of allocated the pinned memory. In the simple first column copy situation, the cost is outweighed by a simple paged memory allocation and coalescing the columns.

References

[1] Aaron Grattafiori et al. *The Llama 3 Herd of Models*. 2024. arXiv: 2407.21783 [cs.AI]. URL: https://arxiv.org/abs/2407.21783.

[2] Dan Hendrycks and Kevin Gimpel. *Gaussian Error Linear Units (GELUs)*. 2023. arXiv: 1606.08415 [cs.LG]. URL: https://arxiv.org/abs/1606.08415.

[3] Noam Shazeer. *GLU Variants Improve Transformer*. 2020. arXiv: 2002.05202 [cs.LG]. URL: https://arxiv.org/abs/2002.05202.

[4] Biao Zhang and Rico Sennrich. "Root Mean Square Layer Normalization". In: *CoRR* abs/1910.07467 (2019). arXiv: 1910.07467. URL: http://arxiv.org/abs/1910.07467.